

EE5203 Microprocessors - 2026-2027 Course Structure

Status: design baseline for iteration. Lecture 1 is the reference package; later lecture materials will be revised after that package is approved.

Course Design Statement

EE5203 develops embedded-system competence through a C-first STM32 pathway. Students first build reliable Embedded C foundations, then learn enough register-level programming to understand the machine, and finally use STM32 HAL to implement increasingly complex peripherals and integrated systems.

The course follows four principles:

1. **Embedded C receives four full lectures.** Later material assumes demonstrated competence in functions, arrays, bitwise operations, pointers, structures, memory, and `volatile`.
2. **Register literacy, HAL fluency.** GPIO and simple register operations are implemented directly. Complex peripherals are implemented with HAL while students inspect and interpret the important registers.
3. **Laboratories fade their scaffolding.** Early labs teach the workflow; later labs provide specifications, references, tests, and individual checkoffs.
4. **The project starts early and grows through milestones.** Students see the destination in L01, form teams in L05, and demonstrate working subsystems throughout the term.

Learning Outcomes

By the end of the course, a successful student can:

1. Write, compile, debug, and test Embedded C programs.
2. Explain the Cortex-M memory, register, and exception model.
3. Configure GPIO, timers, PWM, ADC, UART, and a synchronous serial peripheral using an appropriate combination of CMSIS register access and STM32 HAL.
4. Design interrupt-driven, non-blocking embedded applications.
5. Interpret datasheets, reference manuals, timing diagrams, compiler messages, debugger state, and peripheral registers as engineering evidence.
6. Integrate, test, document, demonstrate, and individually explain an STM32-based embedded system.

Stable Instructional Sequence

Lecture IDs describe reusable instructional order, not calendar weeks.

ID	Topic	Central student performance	Abstraction policy	Project connection
L01	Embedded systems, C toolchain, and debugging	Build, flash, debug, and modify Blinky	HAL used as a working black box; inspect the workflow	See example projects and identify possible inputs/outputs
L02	Values, decisions, loops, and arrays	Process a fixed sensor-data array in one C file	Ordinary C plus debugger	Identify project input and processing data
L03	Functions, arrays, strings, and modular C	Decompose the sensor program into functions and files	Ordinary C plus debugger	Identify project processing functions
L04	Pointers, structures, bits, registers, and GPIO	Explain masks and control PA5 through CMSIS registers	Carefully scaffolded direct register access	Draft individual project idea canvas
L05	GPIO and hardware abstraction	Rebuild GPIO with HAL and compare the two interfaces	Register-versus-HAL comparison	Teams form; project launched
L06	HAL GPIO and systematic debugging	Rebuild GPIO with HAL and compare abstractions	Full register-versus-HAL comparison	Proposal and block diagram
L07	Interrupts, NVIC, and state-based design	Implement and explain an event-driven application	HAL implementation with register inspection	Technical design review

ID	Topic	Central student performance	Abstraction policy	Project connection
L08	Timers and non-blocking time	Generate periodic events without blocking delays	One minimal register demonstration; HAL core work	First peripheral milestone
L09	PWM, output compare, and input capture	Generate and measure timed signals	HAL plus timer-register inspection	Actuator or measurement milestone
L10	ADC and sensor acquisition	Acquire, scale, and validate analog data	HAL plus status/configuration inspection	Data-acquisition milestone
L11	UART and PC communication	Implement logging and command input	HAL; interpret baud/frame settings and flags	Communication milestone
L12	I2C and SPI fundamentals	Interface one synchronous serial device deeply	HAL; read timing diagrams and status evidence	Sensor/display integration
L13	System integration and project clinic	Integrate peripherals, diagnose faults, and test requirements	Student selects appropriate abstraction	Working integrated prototype
L14	Demonstration, oral defense, and reflection	Demonstrate, test, and individually explain the system	Full-system reasoning	Final project assessment

Assembly/disassembly is supporting architectural literacy, integrated into L01, L04, and L05. DMA, TFT displays, stepper motors, and similar advanced devices are optional project or extension tracks rather than simultaneous core requirements.

Assessment Baseline

This model preserves the 2025-2026 grade-workbook weights while improving what each category measures. It must be reconciled with Akademik Paket and CATS before publication.

Category	Weight	Internal structure
Midterm	25%	15% written reasoning + 10% individual toolchain practical (VS Code)
Project	20%	Proposal, subsystem milestones, integration, final demo, report, and oral defense
Laboratory	25%	15% individual lab checkoffs + 10% pre-lab quizzes/homework
Final	30%	Applied written assessment with an approved reference/datasheet policy

The term **Laboratory** must refer to directly observed laboratory performance, not merely an average of quizzes and homework.

Laboratory Progression

Stage	Lectures	Material supplied	Student responsibility
Fully guided	L01-L02	Setup detail, screenshots where essential, starter code, frequent checkpoints	Learn the workflow and make a small individual change
Guided completion	L03-L05	Circuit/configuration information and meaningful TODOs	Complete core C/register expressions and demonstrate them

Stage	Lectures	Material supplied	Student responsibility
Structured independence	L06-L08	Requirements, references, acceptance tests, minimal starter project	Select configuration and program structure
Specification driven	L09-L12	Functional specification, constraints, test cases, documentation links	Create, implement, debug, and verify the solution
Integration	L13-L14	System requirements and milestone rubric	Integrate, test, document, demonstrate, and defend

Every lab uses the same cycle:

Prepare -> 15-minute concept brief -> Foundation checkpoint ->
Independent core task -> Individual modification -> Oral check

Project Timeline

Point	Activity	Evidence
L01	Show achievable examples	No graded submission
L03	Explore hardware and possible functions	Individual idea list
L04	Publish framework and scope model	One-page idea canvas
L05	Form teams and launch	Team and preliminary topic
L06	Proposal checkpoint	Problem, block diagram, components, minimum viable system
L07	Design review	Pins, peripherals, data flow, responsibilities, risks
L08-L09	First hardware milestone	One input or output peripheral working
L10	Data-acquisition milestone	Acquired and validated sensor data

Point	Activity	Evidence
L11	Communication milestone	UART logging or command interface
L12	Integration milestone	Input, processing, output, and communication connected
L13	Readiness review	Working prototype and test evidence
L14	Final assessment	Demonstration, report, presentation, and individual oral defense

Core Material Package

Each lecture has one landing-page section with:

1. **Prepare:** objectives, short preparation task, and pre-lab quiz.
2. **Learn:** core theory deck and two-page study sheet.
3. **Practise:** 30-45 minute worksheet and delayed worked solutions.
4. **Lab:** lab guide, reference page, starter files when justified, checkoff requirements, and submission link.
5. **Review:** mastery checklist and 5-8 exam-style questions.

Material limits:

- Core theory deck: target 28-32 slides; hard maximum 35.
- Laboratory opening deck/briefing: maximum 12-15 slides when slides are needed.
- Study sheet: maximum two pages.
- Student lab guide: maximum two pages and normally 700-900 words.
- Practice worksheet: 30-45 minutes of work.
- Pre-lab quiz: five questions, two attempts, highest score recorded.
- Complete solutions remain under `private/solutions/` until their release time.

Instructor-side, each lecture also carries an instructor guide and a **one-page TA prep sheet** (`private/instructor/L##/TA_Prep_L##.md`): prep checklist (the TA does the lab on hardware before the session), expected-evidence quick reference, a symptom-to-first-question triage table, rubric summary, oral spot-checks, and boundaries (what not to reveal). General TA policy — the evidence-first triage method, checkoff etiquette, solutions-access rules — lives once in `private/instructor/TA_HANDBOOK.md`; prep sheets stay lecture-specific.

Exit Tickets and In-Lecture Pulse Checks (Google Forms)

Every theory deck ends with an exit ticket delivered as a **Google Forms quiz** reached by a QR code on the closing slide (3 questions from the deck's exit question, plus a required student-number field). Standard settings, proven with the L02 template form:

- Quiz mode; release grades immediately; show missed questions and points but **hide correct answers** (limits leakage from early finishers' screens). The key is revealed in the next lecture's retrieval segment.
- Collect **verified** email addresses (Google sign-in required) and limit to one response per account. Students sign in with personal Gmail, so the student number is the roster key; the verified email links one person's tickets across weeks.
- Ungraded: the tickets inform teaching (which misconception is alive in the room) and feed the retrieval slides — they never enter the grade workbook.
- Production: copy the previous lecture's form in Drive (settings and structure carry over), swap the questions, publish, regenerate the QR (`segno` one-liner into `lectures/L##_*/assets/exit_ticket_qr.svg`). Form URLs are logged per lecture in `MATERIALS_STATUS.md`.
- Graded pre-lab quizzes stay in CATS Tests & Quizzes, not Google Forms — anything touching the Laboratory grade needs the official LMS record and the two-attempt policy. Quiz sources (`*PreLab_Quiz*.qmd`) are master copies for CATS only: they are excluded from the rendered site (`_quarto.yml`) and must never appear on the public GitHub Pages site.

Theory Deck Pattern

Section	Typical slides
Opening problem and outcomes	2-3
Retrieval from previous lecture	3-4
Concept block 1	5-6
Guided example/live coding	4-5
Concept block 2	5-6
Student practice/discussion	4-5
Common mistakes/debugging	2-3
Summary, exit question, lab preview	2-3

One core slide normally serves one instructional purpose. Full programs, alternative implementations, long setup sequences, detailed peripheral tables, bonus topics, and complete solutions belong in references, source files, or appendices rather than the planned teaching sequence.

Register/HAL Boundary

- Use CMSIS symbolic register structures, not repeated raw numeric addresses.
- Implement RCC enable and GPIO fundamentals directly at register level.
- Use HAL for interrupts, timers, PWM, input capture, ADC, UART, I2C/SPI, and advanced integration after explaining the hardware model.
- Inspect important registers after HAL configuration.
- Declare the ownership model of every exercise: bare metal, HAL, or one controlled post-HAL register modification.
- Keep LL as an optional industry reference rather than a third core dialect.

Iteration Rule

L01 is the reference implementation for navigation, document length, visual hierarchy, preparation, practice, lab checkoffs, and instructor support. Revise L01 until approved before mechanically applying its pattern to L02-L14.